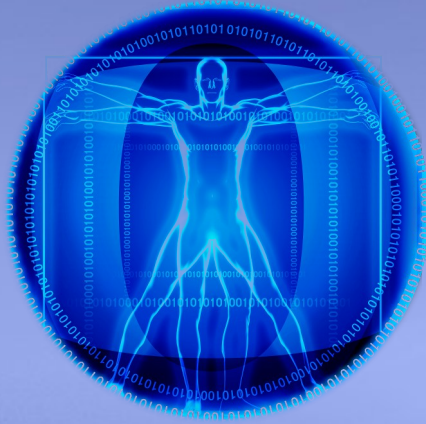


Cybernetics Oriented Programming (CYBOP)

GNU GPLv3

Christian Heller
<christian.heller@tuxtax.de>

<http://www.cybop.org/>



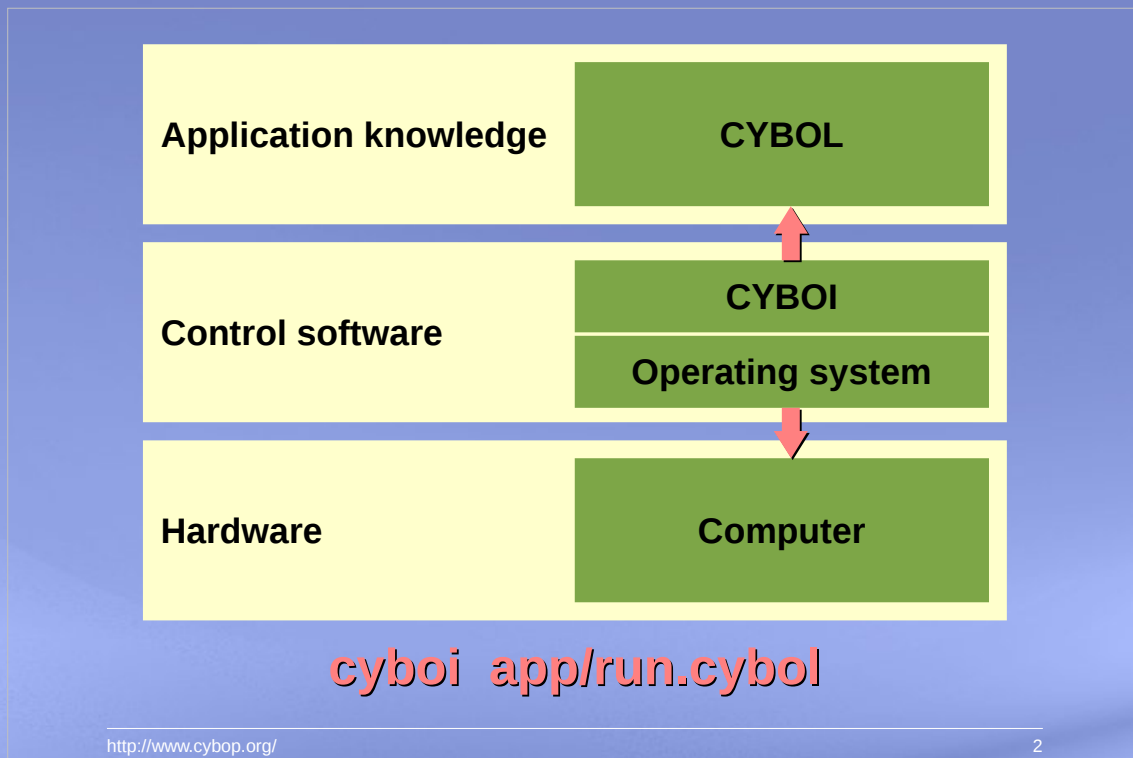
1

Liebe Gleichgesinnte,

ich möchte heute ein freies Softwareprojekt namens CYBOP vorstellen, an welchem ich seit nunmehr 14 Jahren arbeite.

Den Begriff „Kybernetik-orientiert“ habe ich gewählt, da viele Ideen Informatik-fremder Wissenschaftsgebiete in das Projekt eingeflossen sind.

Beispiele finden sich im Anhang dieser Folien.



Das Projekt besteht aus:

- der XML-basierten Sprache CYBOL
- dem in C geschriebenen Interpreter CYBOI

Der CYBOI-Interpreter läuft als aktiver Prozess und verarbeitet passive CYBOL-Dateien.

Der Programmstart erfolgt wie bei jeder anderen interpretierten Sprache.

Criterion	Java world	CYBOP world
Language	Java	CYBOL (XML-based)
Interpreter	Java Virtual Machine (JVM)	CYBOI (written in C)
Theory	Object Oriented Programming (OOP)	CYBOP (knowledge schema)

<http://www.cybop.org/> 3

Zum besseren Verständnis hier ein Vergleich der Welt der Programmiersprache Java mit CYBOP.

Es gibt:

- eine Sprache
- einen Interpreter
- eine zu Grunde liegende Philosophie

Java Class

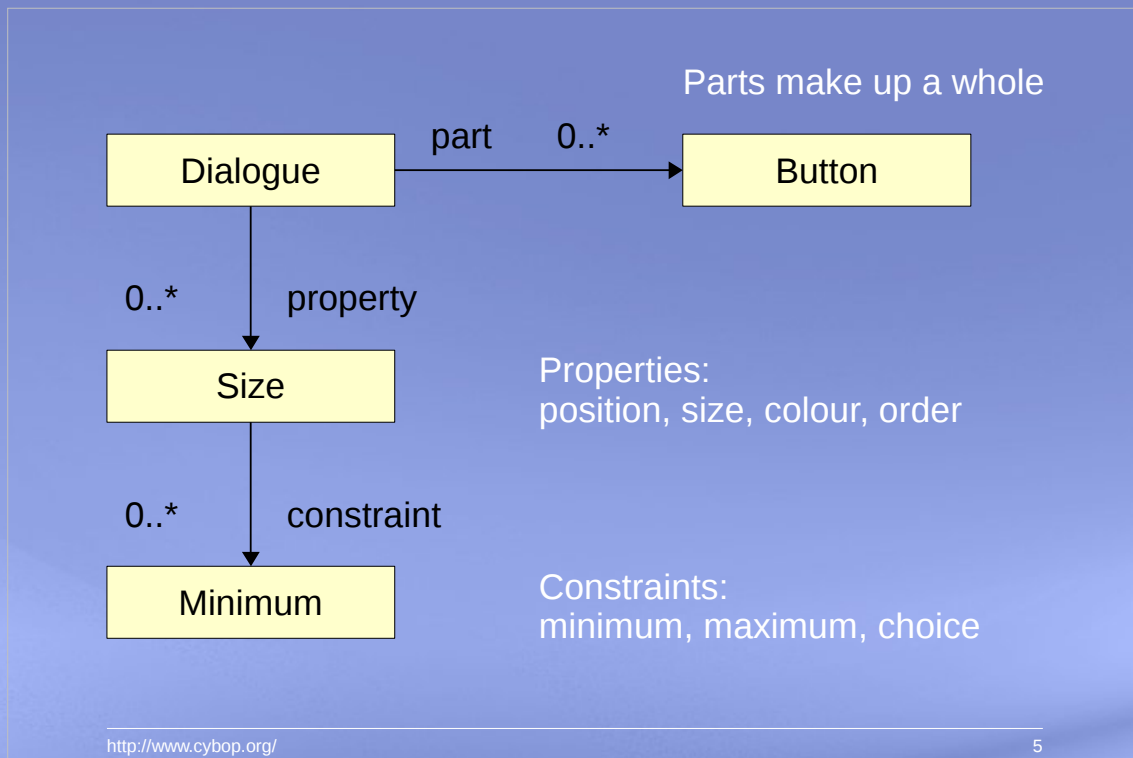
```
public class Dialogue {  
    // Structural attributes  
    private String title;  
    private Toolbar toolbar;  
    private Button button;  
  
    // Additional properties  
    private Point position;  
    private Dimension size;  
    private Color background;  
  
    // Methods  
    public void initialise(Object o);  
    public void process(Event e);  
}
```

Beispielhaft soll nun das Konzept eines grafischen Fensters dargestellt werden.

Zunächst als Objekt-orientierte Klasse, in Java-Syntax.

Es gibt:

- Strukturinformationen
- ergänzende Eigenschaften
- Methoden



Die Vermischung von Strukturinformationen (im Sinne des Mikro-/Makrokosmos) und zusätzlichen Meta-Eigenschaften in quasi allen herkömmlichen Programmiersprachen wurde bisher kaum problematisiert.

Doch ist es erst ihre Unterscheidung mittels Doppelhierarchie, die es ermöglicht, Anwendungsmodelle uniform zu verarbeiten und Kommunikation zu erleichtern.

State Template "dialogue.cybol" (a structure in space)

```
<node>
  <node name="title" channel="inline" format="text/plain" model="Test"/>
  <node name="toolbar" channel="file" format="element/part" model="path/toolbar.cybol"/>
  <node name="button" channel="file" format="element/part" model="path/button.cybol">
    <node name="position" channel="inline" format="number/integer" model="5,5"/>
    <node name="size" channel="inline" format="number/integer" model="600,400"/>
    <node name="background" channel="inline" format="colour/terminal" model="red"/>
  </node>
</node>
```

Logic Template "initialise.cybol" (an algorithm in time)

```
<node>
  <node name="build_menu" channel="inline" format="flow/sequence" model="">
    <node name="model" channel="inline" format="path/knowledge" model=".app.logic.menu"/>
  </node>
  <node name="build_panel" channel="inline" format="flow/sequence" model="">
    <node name="model" channel="inline" format="path/knowledge" model=".app.logic.panel"/>
  </node>
  <node name="display" channel="inline" format="communicate/send" model="">
    <node name="channel" channel="inline" format="meta/channel" model="terminal"/>
    <node name="encoding" channel="inline" format="meta/encoding" model="utf-8"/>
    <node name="language" channel="inline" format="meta/language" model="message/tui"/>
    <node name="format" channel="inline" format="meta/format" model="element/part"/>
    <node name="message" channel="inline" format="path/knowledge" model=".app.dialogue"/>
    <node name="clear" channel="inline" format="logicvalue/boolean" model="true"/>
  </node>
</node>
```

<http://www.cybol.org/>

6

Die Sprache CYBOL stellt dafür eine passende Struktur bereit.

Ihr Vokabular umfasst lediglich ein XML-Tag und vier XML-Attribute.

Essentiell ist die Trennung zwischen Raum und Zeit, das heißt Struktur und Ablauf.

Dabei gilt aber für beide dieselbe Syntax.

CYBOL Features

language including knowledge tree

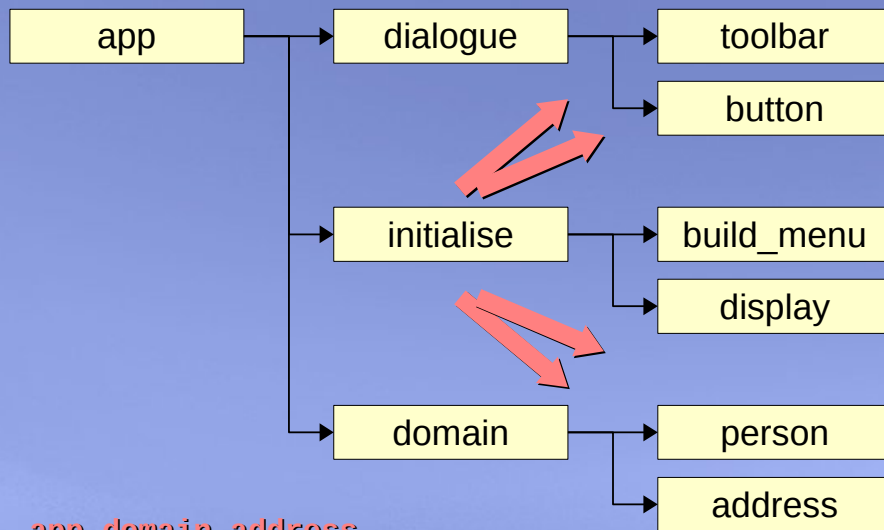
structure based on double-hierarchy pattern

communication using send/receive with cli/tui/gui/wui

usage for standard applications and complex scripting

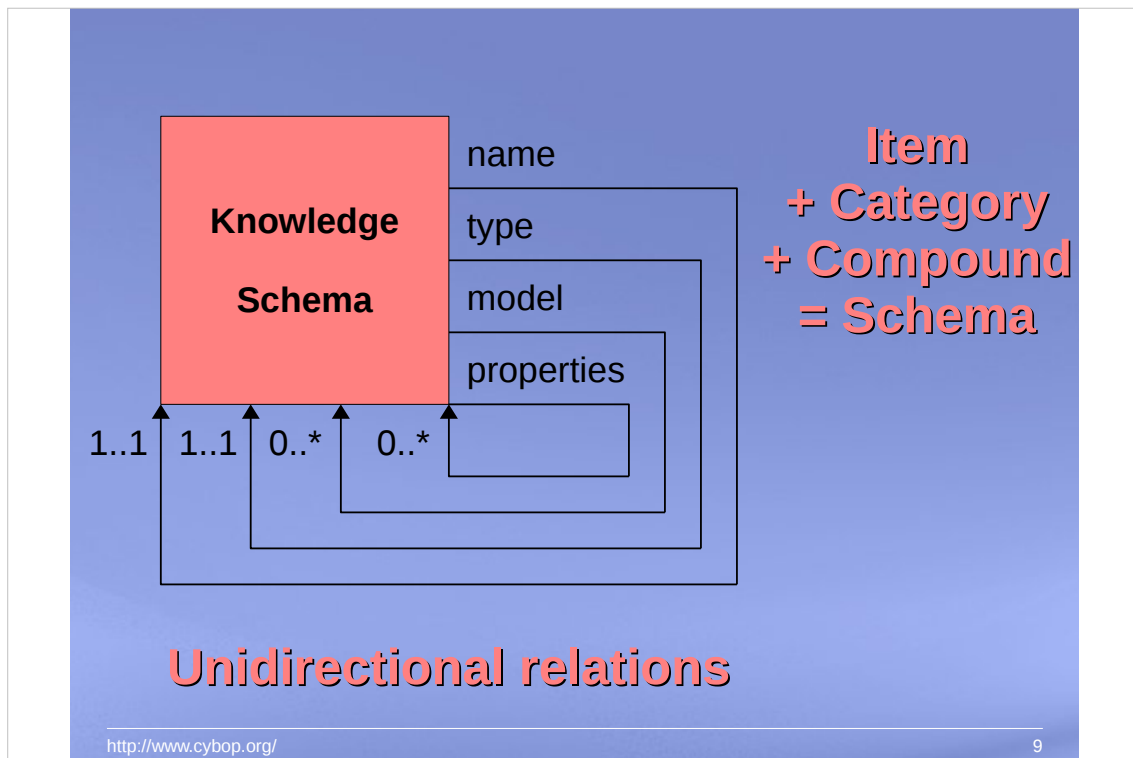
understanding possible for analysts/non-programmers

.app.dialogue.button#position



.app.domain.address

Der Datenbaum zur Laufzeit einer Anwendung enthält Logikmodelle (Operationen über die Zeit), welche Zustandsmodelle manipulieren.



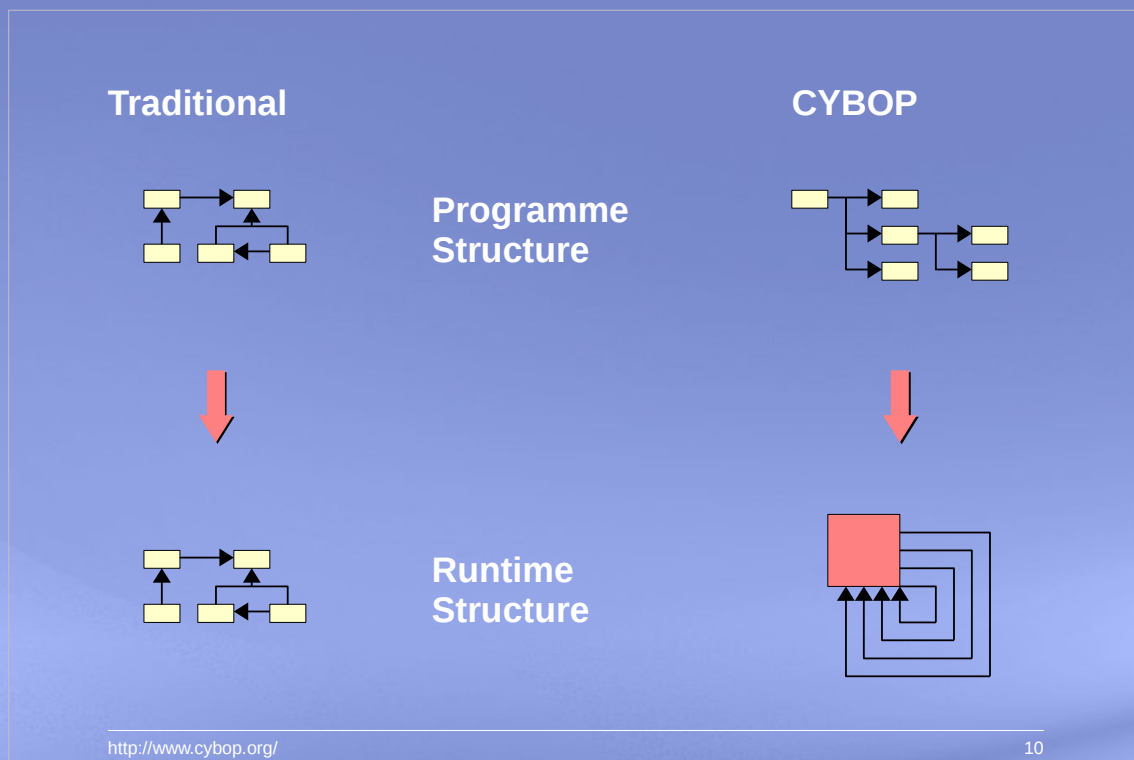
Die einzige verbleibende Struktur zur Speicherung von Daten im Arbeitsspeicher (RAM) ist das CYBOP-Schema.

Es hat rekursive Verbindungen, wodurch eine Baumstruktur entsteht.

Ein Knoten hat:

- eindeutigen Namen (ähnlich einer Web-Seite)
- einen Typen
- optional Kindknoten
- optional Meta-Eigenschaften

Der CYBOP-Interpreter nutzt das Schema in dieser Form.



Dieser Vergleich zeigt Programmstrukturen zur Entwurfszeit und zur Laufzeit.

Klassische Programmiersprachen (prozedural wie auch OO) verdrahten Daten fest.

CYBOP nutzt sein Schema als universelle Speicherstruktur und ist dadurch sehr flexibel.

Auch ist damit relativ leicht implementierbar:

- Garbage Collection (GC)
- Sicherheitsprüfungen

CYBOI Features

one universal knowledge schema

representing the only unified container structure

one main event loop

many communication channels (terminal, xcb, win32, socket)

full i18n

Addendum

Inter-Disciplinary Ideas

Philosophy: Body & Mind = active system & passive data

Biology: Cell & DNA = system & application knowledge

Psychology: Thinking = item + category + compound

Physics: Space & Time = state-structure & logic-workflow

Solved Problems

falsified container content - due to container inheritance
fragile base class problem - due to polymorphism
untraceable data manipulation - due to global data access
circular references - due to bidirectional dependencies
bidirectional dependencies - due to reflective architectures
strong coupling - due to bundling of attributes and methods
difficult maintenance - due to high complexity

Characteristics

- long-life software system due to strict separation of application knowledge and system software
- less dependencies due to representation of structural- and meta data in form of a double hierarchy
- flexible architecture due to unified schema as the only remaining static memory structure
- clean knowledge model code due to tree structure with unidirectional relations
- reflexive meta programming due to changeable logic
- universal communication due to star-like (not layer-like) translation architecture
- easy handling due to one syntax describing structures (space) and algorithms (time) in the same way
- superfluous implementation phase due to direct interpretation of knowledge



Traditional



CYBOP

models suffer from complexity	one schema memory structure
strong coupling/dependencies	directed acyclic graph/tree
inflexible	easily extensible
difficult to maintain	long-life software system